

Modules — বড় JavaScript Project-কে ছোট, Organized ও Reusable File-এ ভাগ করা

প্রফেশনাল সফটওয়্যার ইঞ্জিনিয়ারিংয়ে কোনো বড় কোডবেস একটিমাত্র ফাইলে রাখা অসম্ভব। কোডবেসকে পরিষ্কার, রিইউজেবল, সহজে টেস্টেবল এবং মেইনটেইনেবল রাখতে জাভাস্ক্রিপ্টের অফিসিয়াল মডিউল স্ট্যান্ডার্ড **ES Modules (ESM)** ব্যবহার করা হয়।

Single Responsibility File → export (Named / Default) → Module Dependency Graph → import / Dynamic import()

React, Vue, Next.js বা Node.js যেকোনো আধুনিক ফ্রন্টএন্ড ফ্রেমওয়ার্কের ভিত্তিই হলো এই মডিউল সিস্টেম। 🔥

PART 01 — MODULE ARCHITECTURE & THE ES MODULES STANDARD (20.1 – 20.8)

20.1 Module কী?

Module হলো JavaScript code-এর একটি আলাদা file, যেখানে নির্দিষ্ট functionality রাখা হয় এবং প্রয়োজন হলে অন্য file-এ `export` / `import` করা যায়।

সহজভাবে:

```
একটা বড় Project
├──
├──
└──
  ├── auth.js
  ├── cart.js
  └── utils.js
```

প্রতিটি file নিজের কাজ করবে।

20.2 Real-life Example

ধরো তুমি একটি E-commerce website বানাচ্ছে।

সব code এক file-এ:

```
app.js
├── Login
├── Register
├── Products
├── Cart
├── Payment
├── Validation
└── Utility
```

এটা বড় হলে manage করা কঠিন।

তার বদলে:

```
project/
├── index.html
├── js/
│   ├── app.js
│   ├── auth.js
│   ├── cart.js
│   ├── products.js
│   └── utils.js
```

এটাই modular architecture।

20.3 Module-এর প্রধান সুবিধা

TEXT

1. Code organization
2. Reusability
3. Maintainability
4. Encapsulation
5. Namespace pollution কমানো
6. Dependency management
7. Team collaboration সহজ

20.4 Normal JavaScript File

ধরো:

20.4 JAVASCRIPT MODULE

ESM

```
function add(a, b) {
  return a + b;
}
```

```
console.log(add(10, 20));
```

TEXT

30

এখন `add()` অন্য file-এ ব্যবহার করতে চাই।

এখানেই module দরকার।

20.5 ES Modules

Modern JavaScript-এ module system হলো:

TEXT

ES Modules
বা
ESM

এর প্রধান keyword:

20.5 JAVASCRIPT MODULE

ESM

`export`
`import`

20.6 প্রথম Module Example

 MATH.JS

20.6 `MATH.JS`

ESM

```
export function add(a, b) {  
  return a + b;  
}
```

 APP.JS

20.6 `APP.JS`

ESM

```
import { add } from "./math.js";  
  
console.log(add(10, 20));
```

TEXT

30

এখানে:

```
math.js  
↓  
export  
↓  
add()  
↓  
app.js  
↓  
import
```

20.7 HTML-এ Module Load করা

Browser-এ ES Module ব্যবহার করতে:

20.7 HTML

MARKUP

```
<script  
  type="module"  
  src="./js/app.js">  
</script>
```

★ খুব গুরুত্বপূর্ণ

20.7 HTML

MARKUP

type="module"

না দিলে browser এটাকে normal script হিসেবে treat করবে এবং `import/export` কাজ করবে না।

20.8 Complete Folder Structure

```
project/
├── index.html
└── js/
    ├── app.js
    └── math.js
```

INDEX.HTML

20.8 HTML

MARKUP

```
<!DOCTYPE html>
<html>
<head>
  <title>JS Modules</title>
</head>

<body>

  <h1>JavaScript Modules</h1>

  <script
    type="module"
    src="./js/app.js">
  </script>

</body>
</html>
```

MATH.JS

20.8 'MATH.JS'

ESM

```
export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}
```

APP.JS

20.8 'APP.JS'

ESM

```
import {
  add,
  subtract
} from "./math.js";

console.log(add(10, 5));
console.log(subtract(10, 5));
```

TEXT

```
15
5
```

20.9 Named Export

একাধিক function export করতে:

20.9 JAVASCRIPT MODULE ESM

```
export function add(a, b) {  
  return a + b;  
}  
  
export function multiply(a, b) {  
  return a * b;  
}  
  
export const PI = 3.14159;
```

Import:

20.9 JAVASCRIPT MODULE ESM

```
import {  
  add,  
  multiply,  
  PI  
} from "./math.js";
```

20.10 Export at the Bottom

Function-এর সামনে `export` না দিয়ে শেষেও export করা যায়।

20.10 JAVASCRIPT MODULE ESM

```
function add(a, b) {  
  return a + b;  
}  
  
function subtract(a, b) {  
  return a - b;  
}  
  
export {  
  add,  
  subtract  
};
```

Import:

20.10 JAVASCRIPT MODULE ESM

```
import {  
  add,  
  subtract  
} from "./math.js";
```

দুটো approach valid।

20.11 Named Import

যদি:

```
20.11 JAVASCRIPT MODULE ESM
export function add() {}

export function subtract() {}
```

তাহলে:

```
20.11 JAVASCRIPT MODULE ESM
import {
  add,
  subtract
} from "../math.js";
```

এটাই named import।

20.12 Import Rename

ধরো:

```
20.12 JAVASCRIPT MODULE ESM
import {
  add as sum
} from "../math.js";

console.log(sum(10, 20));
```

TEXT

30

এখানে:

```
add → sum
```

নামে locally ব্যবহার করছি।

20.13 Export Rename

এভাবেও করা যায়:

```
20.13 JAVASCRIPT MODULE ESM
function add(a, b) {
  return a + b;
}

export {
  add as sum
};
```

তখন import:

```
20.13 JAVASCRIPT MODULE ESM
import {
  sum
} from "../math.js";
```

20.14 Default Export

একটি module-এর একটি **default export** থাকতে পারে।

20.14 JAVASCRIPT MODULE

ESM

```
export default function add(a, b) {  
  return a + b;  
}
```

Import:

20.14 JAVASCRIPT MODULE

ESM

```
import add from "./math.js";
```

এখানে `{}` লাগে না।

20.15 Default Export Rename করা যায়

ধরো:

20.15 JAVASCRIPT MODULE

ESM

```
export default function add(a, b) {  
  return a + b;  
}
```

Import:

20.15 JAVASCRIPT MODULE

ESM

```
import sum from "./math.js";
```

এটাও valid।

কারণ default export import করার সময় local নাম তুমি নিজেই দিতে পারো।

20.16 Named vs Default Export

বিষয়	Named Export	Default Export
Syntax	<code>export { add }</code>	<code>export default add</code>
Import	<code>{ add }</code>	<code>add</code>
Multiple	Yes	One default per module
Rename	<code>as</code> দিয়ে	Import-এই নাম দেওয়া যায়

Example:

20.16 JAVASCRIPT MODULE

ESM

```
// Named
export {
  add,
  subtract
};
```

Import:

20.16 JAVASCRIPT MODULE

ESM

```
import {
  add,
  subtract
} from "./math.js";
```

Default:

20.16 JAVASCRIPT MODULE

ESM

```
export default add;
```

Import:

20.16 JAVASCRIPT MODULE

ESM

```
import add from "./math.js";
```

20.17 একই Module-এ Named + Default

এটাও করা যায়।

20.17 JAVASCRIPT MODULE

ESM

```
export const PI = 3.14159;

export function multiply(a, b) {
  return a * b;
}

export default function add(a, b) {
  return a + b;
}
```

Import:

20.17 JAVASCRIPT MODULE

ESM

```
import add, {
  PI,
  multiply
} from "./math.js";
```

20.18 Namespace Import

সব named export একসাথে object-এর মতো ব্যবহার করতে:

```
20.18 JAVASCRIPT MODULE ESM
import * as math
from "./math.js";

console.log(
  math.add(10, 20)
);

console.log(
  math.multiply(5, 4)
);
```

এখানে:

```
TEXT
math.add()
math.multiply()
math.PI
```

ব্যবহার করা যাবে।

20.19 Namespace Import-এর Real-life Use

ধরো:

```
utils.js
├── formatDate
├── formatCurrency
├── capitalize
└── debounce
```

Import:

```
20.19 JAVASCRIPT MODULE ESM
import * as utils
from "./utils.js";

utils.formatDate();
utils.formatCurrency();
utils.capitalize();
```

এতে related functions একটি namespace-এর মধ্যে থাকে।

20.20 Side-effect Import

কখনো শুধু module execute করতে import করা হয়।

```
20.20 JAVASCRIPT MODULE ESM
import "./analytics.js";
```

এখানে কোনো function import করছি না।

শুধু:

```
analytics.js
↓
execute
```

হবে।

20.21 Module Scope

এটি খুব গুরুত্বপূর্ণ।

Normal script-এ global variable তৈরি হতে পারে।

Module-এ:

20.21 JAVASCRIPT MODULE

ESM

```
const username = "Shariar";
```

এই variable automatically global scope-এ চলে যায় না।

অন্য module সরাসরি:

20.21 JAVASCRIPT MODULE

ESM

```
console.log(username);
```

করতে পারবে না।

যদি দরকার হয়:

20.21 JAVASCRIPT MODULE

ESM

```
export const username = "Shariar";
```

তারপর:

20.21 JAVASCRIPT MODULE

ESM

```
import {  
  username  
} from "./user.js";
```

20.22 Modules = Private by Default

ধরো:

20.22 JAVASCRIPT MODULE

ESM

```
const password =  
  "secret123";  
  
export const username =  
  "Shariar";
```

অন্য module `password` access করতে পারবে না।

শুধু exported values accessible।

এটি **encapsulation**-এর basic form।

20.23 Module Dependency

ধরো:

```
app.js  
  ↓  
auth.js  
  ↓  
api.js
```

`app.js` যদি `auth.js` import করে এবং `auth.js` যদি `api.js` import করে, তাহলে dependency chain তৈরি হয়।

```
app.js  
  |  
  ↓  
auth.js  
  |  
  ↓  
api.js
```

Modern bundlers এই dependency graph analyze করে।

20.24 Relative Path

যদি structure:

```
js/  
├─ app.js  
└─ utils.js
```

তাহলে:

20.24 JAVASCRIPT MODULE

ESM

```
import {  
  add  
} from "./utils.js";
```

`./` মানে current directory।

20.25 Parent Directory

Structure:

```
js/  
├─ app.js  
└─ utils/  
    └─ math.js
```

`app.js` থেকে:

20.25 JAVASCRIPT MODULE

ESM

```
import {  
  add  
} from "../utils/math.js";
```

আর nested file থেকে parent-এ যেতে:

```
TEXT  
  
../
```

ব্যবহার হয়।

20.26 File Extension

Browser-based native ES Modules-এ:

20.26 JAVASCRIPT MODULE

ESM

```
import {  
  add  
} from "./math.js";
```

`.js` extension explicitly দেওয়া ভালো/প্রয়োজনীয়।

Bundler-based project যেমন Vite/webpack-এ tooling কিছু path handling করতে পারে, কিন্তু native browser module বুঝতে এই বিষয়টি জানা জরুরি।

20.27 Module Script-এর Important Features

20.27 HTML

MARKUP

```
<script  
  type="module"  
  src="./app.js">  
</script>
```

Module scripts:

- strict mode semantics ব্যবহার করে
- module scope রাখে
- `import/export` support করে
- dependency অনুযায়ী modules load করে
- browser-এ সাধারণত deferred behavior-এর মতো execute হয়

20.28 Strict Mode

Module code automatically strict mode semantics-এ চলে।

তাই সাধারণত:

```
20.28 JAVASCRIPT MODULE
```

ESM

```
"use strict";
```

আলাদাভাবে লেখার দরকার নেই।

Example:

```
20.28 JAVASCRIPT MODULE
```

ESM

```
x = 10;
```

Module-এ undeclared variable assignment error দেবে।

20.29 Module-এর Top-Level `this`

Module-এর top-level `this`:

```
20.29 JAVASCRIPT MODULE
```

ESM

```
console.log(this);
```

Browser ES module-এ সাধারণত:

```
TEXT
```

```
undefined
```

পাওয়া যায়।

এটি normal classic script-এর behavior থেকে আলাদা।

PART 04 — DYNAMIC IMPORTS, RE-EXPORTING & PROJECT ARCHITECTURE (20.30 – 20.43)

20.30 Dynamic Import

সবসময় module শুরুতেই load করতে হবে এমন নয়।

Runtime-এ module load করা যায়:

```
20.30 JAVASCRIPT MODULE
```

ESM

```
const module =  
  await import("./math.js");  
  
console.log(  
  module.add(10, 20)  
);
```

এটাকে:

```
TEXT
```

```
Dynamic Import
```

বলে।

20.31 Dynamic Import কেন দরকার?

ধরো user একটা বিশেষ feature click করল:

```
User clicks  
"Advanced Editor"  
↓  
Load editor.js
```

শুরুতেই editor code load না করে প্রয়োজনের সময় load করা যায়।

এতে বড় application-এর initial loading optimize করা সম্ভব।

20.32 Dynamic Import Without `await`

20.32 JAVASCRIPT MODULE

ESM

```
import("./math.js")
  .then(module => {

    console.log(
      module.add(10, 20)
    );

  });
```

কারণ dynamic `import()` একটি Promise return করে।

20.33 export

একটি module-এর exports অন্য module থেকে re-export করা যায়।

⚙️ MATH.JS

20.33 `MATH.JS`

ESM

```
export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}
```

⚙️ INDEX.JS

20.33 `INDEX.JS`

ESM

```
export * from "./math.js";
```

এখন অন্য file:

20.33 `INDEX.JS`

ESM

```
import {
  add,
  subtract
} from "./index.js";
```

এটাকে **barrel/re-export pattern**-এর basic example হিসেবে দেখতে পারো।

20.34 Re-export with Rename

20.34 JAVASCRIPT MODULE

ESM

```
export {
  add as sum
} from "./math.js";
```

এখন:

20.34 JAVASCRIPT MODULE

ESM

```
import {
  sum
} from "./index.js";
```

20.35 Default Re-export

20.35 JAVASCRIPT MODULE

ESM

```
export {  
  default  
} from "./math.js";
```

অথবা:

20.35 JAVASCRIPT MODULE

ESM

```
export {  
  default as calculate  
} from "./math.js";
```

20.36 Practical Project Structure

একটি ছোট e-commerce project:

```
project/  
├── index.html  
├── css/  
│   └── style.css  
└── js/  
    ├── app.js  
    ├── api.js  
    ├── auth.js  
    ├── cart.js  
    ├── products.js  
    ├── storage.js  
    └── utils.js
```

Responsibilities:

```
app.js  
→ application entry point  
  
api.js  
→ API communication  
  
auth.js  
→ login/logout  
  
cart.js  
→ shopping cart  
  
products.js  
→ product logic  
  
storage.js  
→ localStorage  
  
utils.js  
→ reusable helper functions
```

এটিই modular thinking!

20.37 Practical `storage.js`

20.37 JAVASCRIPT MODULE

ESM

```
export function saveData(key, value) {  
  
    localStorage.setItem(  
        key,  
        JSON.stringify(value)  
    );  
  
}  
  
export function getData(key) {  
  
    const data =  
        localStorage.getItem(key);  
  
    return data  
        ? JSON.parse(data)  
        : null;  
}  
  
export function removeData(key) {  
  
    localStorage.removeItem(key);  
  
}
```

20.38 app.js

20.38 JAVASCRIPT MODULE

ESM

```
import {  
    saveData,  
    getData,  
    removeData  
} from "../storage.js";  
  
const user = {  
    name: "Sharlar",  
    role: "Developer"  
};  
  
saveData("user", user);  
  
console.log(  
    getData("user")  
);
```

20.38 JAVASCRIPT MODULE

ESM

```
{  
    name: "Sharlar",  
    role: "Developer"  
}
```

20.39 Practical `utils.js`

20.39 JAVASCRIPT MODULE

ESM

```
export function capitalize(text) {  
  
    return (  
        text.charAt(0).toUpperCase() +  
        text.slice(1)  
    );  
  
}  
  
export function calculateDiscount(  
    price,  
    discount  
) {  
  
    return price -  
        (price * discount / 100);  
  
}
```

Use:

20.39 JAVASCRIPT MODULE

ESM

```
import {  
    capitalize,  
    calculateDiscount  
} from "../utils.js";  
  
console.log(  
    capitalize("shariar")  
);  
  
console.log(  
    calculateDiscount(1000, 10)  
);
```

TEXT

```
Shariar  
900
```

20.40 Module + Class

Class-ও export করা যায়।

20.40 JAVASCRIPT MODULE

ESM

```
export class User {  
  
  constructor(name) {  
    this.name = name;  
  }  
  
  greet() {  
    return `Hello ${this.name}`;  
  }  
  
}
```

Import:

20.40 JAVASCRIPT MODULE

ESM

```
import { User }  
from "./User.js";  
  
const user =  
  new User("Shariar");  
  
console.log(  
  user.greet()  
);
```

TEXT

Hello Shariar

20.41 Module + Constant

20.41 JAVASCRIPT MODULE

ESM

```
export const API_URL =  
  "https://example.com/api";
```

Import:

20.41 JAVASCRIPT MODULE

ESM

```
import { API_URL }  
from "./config.js";  
  
console.log(API_URL);
```

Real project-এ configuration আলাদা module-এ রাখা common।

20.42 Module + Async Function

Module-এর ভিতরে:

20.42 JAVASCRIPT MODULE

ESM

```
export async function getUsers() {  
  const response =  
    await fetch("/api/users");  
  
  return response.json();  
}
```

অন্য module:

20.42 JAVASCRIPT MODULE

ESM

```
import { getUsers }  
from "./api.js";  
  
const users =  
  await getUsers();  
  
console.log(users);
```

Modern modules-এ top-level `await` -ও supported environments-এ ব্যবহার করা যায়।

20.43 Top-Level `await`

Module-এর top level-এ:

20.43 JAVASCRIPT MODULE

ESM

```
const response =  
  await fetch("/api/users");  
  
const users =  
  await response.json();  
  
console.log(users);
```

Classic script-এ সাধারণভাবে top-level `await` কাজ করবে না, কিন্তু ES module-এ supported।
এটি asynchronous module loading-এর সাথে related।

PART 05 — ADVANCED MODULE MECHANICS: NODE.JS, COMMONJS VS ESM, LIVE BINDINGS (20.44 – 20.51)

20.44 Module vs Normal Script

বিষয়	Normal Script	ES Module
<code>import/export</code>	✗	✓
Scope	Global/script scope	Module scope
Strict semantics	Not automatically module strict	✓
Reusable dependency system	Limited	✓
<code>type="module"</code>	✗	Browser-এ প্রয়োজন
Top-level <code>await</code>	✗	✓
Dynamic <code>import()</code>	Modern JS-এ available	✓

20.45 Module কেন React-এ গুরুত্বপূর্ণ?

তুমি যখন React ব্যবহার করবে, প্রায় প্রতিটি file module-এর মতো কাজ করবে।

উদাহরণ:

20.45 JAVASCRIPT MODULE

ESM

```
import React from "react";
import Navbar from "../components/Navbar";
import Login from "../pages/Login";
import api from "../services/api";
```

React project মূলত modular architecture-এর উপর heavily নির্ভর করে।

20.46 Node.js-এ Modules

Node.js-এ module system-এর দুটি গুরুত্বপূর্ণ style আছে:

TEXT

CommonJS
ES Modules

CommonJS:

20.46 JAVASCRIPT MODULE

ESM

```
const math =
  require("../math");
```

Export:

20.46 JAVASCRIPT MODULE

ESM

```
module.exports = {
  add
};
```

Modern Node.js-এ ES Modules-ও ব্যবহার করা যায়:

20.46 JAVASCRIPT MODULE

ESM

```
import { add }
from "../math.js";
```

এবং:

20.46 JAVASCRIPT MODULE

ESM

```
export {
  add
};
```

20.47 CommonJS vs ES Modules

CommonJS	ES Modules
<code>require()</code>	<code>import</code>
<code>module.exports</code>	<code>export</code>
Node.js-এর traditional system	JavaScript standard module system
Older Node ecosystem-এ খুব common	Modern JS/browser/Node-এ widely used

Modern full-stack development-এ দুটো system সম্পর্কে জানা useful।

20.48 Circular Dependencies

ধরো:

```
A.js
↓
B.js
↓
A.js
```

অর্থাৎ:

20.48 JAVASCRIPT MODULE

ESM

```
// A.js
import { b } from "./B.js";
```

আর:

20.48 JAVASCRIPT MODULE

ESM

```
// B.js
import { a } from "./A.js";
```

একে **circular dependency** বলে।

এটি কখনো কাজ করতে পারে, কিন্তু **complex initialization** সমস্যা তৈরি করতে পারে।

তাই architecture এমনভাবে design করা ভালো যাতে unnecessary circular dependency না থাকে।

20.49 Modules এবং Encapsulation

ধরো:

20.49 JAVASCRIPT MODULE

ESM

```
let balance = 1000;

export function deposit(amount) {

    balance += amount;

}

export function getBalance() {

    return balance;

}
```

অন্য file:

20.49 JAVASCRIPT MODULE

ESM

```
import {
    deposit,
    getBalance
} from "./bank.js";

deposit(500);

console.log(
    getBalance()
);
```

TEXT

1500

অন্য module সরাসরি:

20.49 JAVASCRIPT MODULE

ESM

```
balance = 999999;
```

করতে পারবে না।

এটাই module-based encapsulation-এর practical ধারণা।

20.50 Live Binding Concept

Modules-এর export সাধারণত **live binding**।

Example:

20.50 JAVASCRIPT MODULE

ESM

```
// counter.js

export let count = 0;

export function increment() {
  count++;
}
```

অন্য file:

20.50 JAVASCRIPT MODULE

ESM

```
import {
  count,
  increment
} from './counter.js';

console.log(count);

increment();

console.log(count);
```

TEXT

```
0
1
```

Imported binding module-এর current exported value-এর সাথে linked থাকে।

তবে importer সাধারণত imported binding নিজে reassign করতে পারে না।

20.51 Module Cache / Reuse

একই module একাধিক জায়গায় import করলে environment সাধারণত module-টিকে বারবার নতুন করে execute না করে module instance reuse করে।

Example:

```
app.js  ┌
        ├──> config.js
cart.js └
```

দুই জায়গা থেকে `config.js` import হলেও একই module instance-এর exports share করা হয়।

এটি application architecture-এ গুরুত্বপূর্ণ।

MUST KNOW 🔥 20.52 Must Know — ES Modules Architecture

এই chapter থেকে অবশ্যই জানতে হবে:

Core

JAVASCRIPT MODULE

ESM

```
export  
import
```

Named

JAVASCRIPT MODULE

ESM

```
export {  
  add  
};
```

JAVASCRIPT MODULE

ESM

```
import {  
  add  
} from "./math.js";
```

Default

JAVASCRIPT MODULE

ESM

```
export default function() {}
```

JAVASCRIPT MODULE

ESM

```
import something  
from "./file.js";
```

Namespace

JAVASCRIPT MODULE

ESM

```
import * as utils  
from "./utils.js";
```

Dynamic

JAVASCRIPT MODULE

ESM

```
import("./module.js");
```

Re-export

JAVASCRIPT MODULE

ESM

```
export * from "./math.js";
```

Browser

HTML

MARKUP

```
<script type="module">
```

কাজ	Syntax
Named export	<code>export { add }</code>
Inline export	<code>export function add(){} </code>
Named import	<code>import { add } from "./math.js" </code>
Rename import	<code>import { add as sum } ... </code>
Default export	<code>export default add </code>
Default import	<code>import add from "./math.js" </code>
Namespace import	<code>import * as math from "./math.js" </code>
Side-effect import	<code>import "./analytics.js" </code>
Dynamic import	<code>import("./math.js") </code>
Re-export	<code>export * from "./math.js" </code>
Browser module	<code><script type="module"> </code>

**Practice 1 — Math Module**

`math.js` তৈরি করো:

TEXT

```
add()  
subtract()  
multiply()  
divide()
```

সবগুলো export করো।

তারপর `app.js` থেকে import করো।

Practice 2 — User Module

`user.js` :

JAVASCRIPT MODULE

ESM

```
export const user = {  
  name: "Shariar",  
  role: "Developer"  
};
```

`app.js` থেকে import করে UI-তে name দেখাও।

Practice 3 — Utility Module

একটি `utils.js` বানিয়ে:

TEXT

```
capitalize()  
calculateDiscount()  
formatPrice()
```

তৈরি করো এবং অন্য file-এ ব্যবহার করো।

Practice 4 — Storage Module

Chapter 18-এর code দিয়ে:

```
storage.js  
↓  
saveData()  
getData()  
removeData()
```

বানাও।

তারপর `app.js` থেকে ব্যবহার করো।

Practice 5 — Mini E-commerce Architecture

এই structure বানাও:

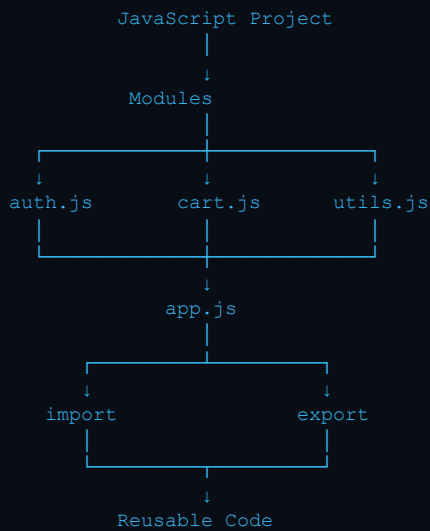
```
js/  
├─ app.js  
├─ products.js  
├─ cart.js  
├─ storage.js  
└─ utils.js
```

Responsibilities:

```
products.js  
→ product data  
  
cart.js  
→ add/remove cart  
  
storage.js  
→ localStorage  
  
utils.js  
→ helper functions  
  
app.js  
→ main application
```

SUMMARY

🎯 20.55 Final Mental Map — Module Organization & Lifecycle

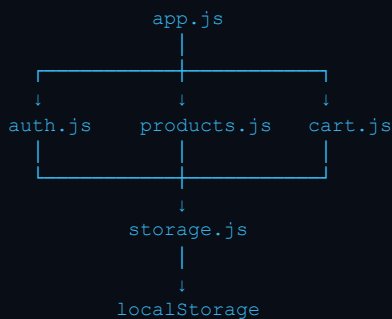


★ Chapter 20-এর সবচেয়ে গুরুত্বপূর্ণ বিষয়

তুমি Full Stack Developer হিসেবে এই concept-টা ভালোভাবে বুঝবে:

একটা বড় JavaScript file
 ↓
 ছোট ছোট modules
 ↓
 প্রতিটি module-এর নির্দিষ্ট কাজ
 ↓
 export
 ↓
 import
 ↓
 Main Application

একটি real project:



আর React/Node.js project-এ তুমি যে:

JAVASCRIPT MODULE

ESM

```
import ...
export ...
```

প্রায় প্রতিদিন দেখবে—তার foundation হলো এই Chapter 20-এর ES Modules!

Chapter 20 complete.

NEXT CHAPTER

■ Chapter 21 Preview — Asynchronous JavaScript

পরবর্তী অধ্যায়ে আমরা জাভাস্ক্রিপ্টের অন্যতম কোর পিলার **Asynchronous JavaScript** নিয়ে বিস্তারিত শিখব—Event Loop, Call Stack, Callback Hell, Promises (`then` , `catch` , `finally`), Promise Combinators (`all` , `race` , `allSettled` , `any`), এবং আধুনিক `async/await` সিনট্যাক্স।